

35 — Several validation errors at once

Proves · Several validation errors reported together in one response.

Mechanics this scenario turns on

- Validation runs **before** Level 1. A plant that cannot carry its load is rejected there, with a different message and a different code path from the Level 1 rejections.

Panels described below · The deliberately broken input · The response — and its order · What is deliberately not here

Anything not described here — the mechanics above name the step that produced it; `00-ORIENTATION` Part 6 has the full number-to-step index.

Trust · characterisation snapshot, generated from the code. It detects change; it does NOT prove correctness. Figures marked *pending reference verification* have never been checked against anything outside the application.

Read after · scenario 17.

Every other 400 in the suite carries exactly **one** error. This one carries five, which is what tests the *shape* of the failure response rather than any single message.

The deliberately broken input

```
"propulsionPower": 0,           // must be > 0
"seaMargin": 120,              // must be 0...100
"aeCount": 0,                 // must be >= 1
"fuelPrice": 0,               // must be > 0
"sgCapacityPerEngine": 30000   // exceeds the 24 000 kW main engine
```

The response — and its order

```
400
1. Propulsion power must be greater than 0
2. Sea margin must be between 0 and 100
3. Number of aux engines must be at least 1
4. Fuel price must be greater than 0
5. Shaft generator capacity cannot exceed main engine capacity.
```

The order is part of the contract. `ValidationService.ValidateInput` runs five ordered slices — plant and financials, battery, PTI and Excel inputs, operational modes, sail — and only then appends the capacity checks, whose `Error`-severity entries are promoted into this same list.

That is why the shaft-generator error (a promoted capacity *warning*) comes last, after four field-level errors. A refactor that regrouped the checks by topic would reorder this list, the client would render them differently,

and this snapshot would catch it. It is exactly the constraint that shaped the `ValidationService` split in story R-E.

What is deliberately not here

There are more than twenty individual validation messages. Giving each a scenario would triple the suite for no added confidence — `ValidationServiceTests` pins every string and every condition. The golden suite's job is the **response envelope**: status, the list, its order.

Takeaway: one scenario with five errors is worth more than five scenarios with one, because the thing only it can test is the ordering.

KSailCalc manual test scenarios — calculation walkthrough · regenerated 2026-08-10 · numbers verified against commit 559aef2 and unchanged by the backend/client refactors (18 golden snapshots byte-identical)